8. To Detect Variable References, Expand Values, Handle Undefined Variables, Update Tokens, Support Nested Variables, Test Expansion Logic.

Code:



```
ketana_sindhu@KetaZone: ~/
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 300
#define MAX_OUTPUT 500

/* Expand a single variable */
void expandVariable(const char *variable, char *value) {

    const char *result = getenv(variable);

    if (result != NULL) {
        strcpy(value, result);
    }
    else {
        /* Undefined variables become empty */
        value[0] = '\0';
    }
}

/* Expand variables inside a string */
void expandVariables(const char *input, char *output) {

    int i = 0;
    int j = 0;

    while (input[i] != '\0' && j < MAX_OUTPUT - 1) {

        /* Detect variable reference */
        if (input[i] == '$') {

            i++;

            char variable[100];
            int k = 0;

            /* Handle ${VARIABLE} format */
            if (input[i] == '{') {

                i++;

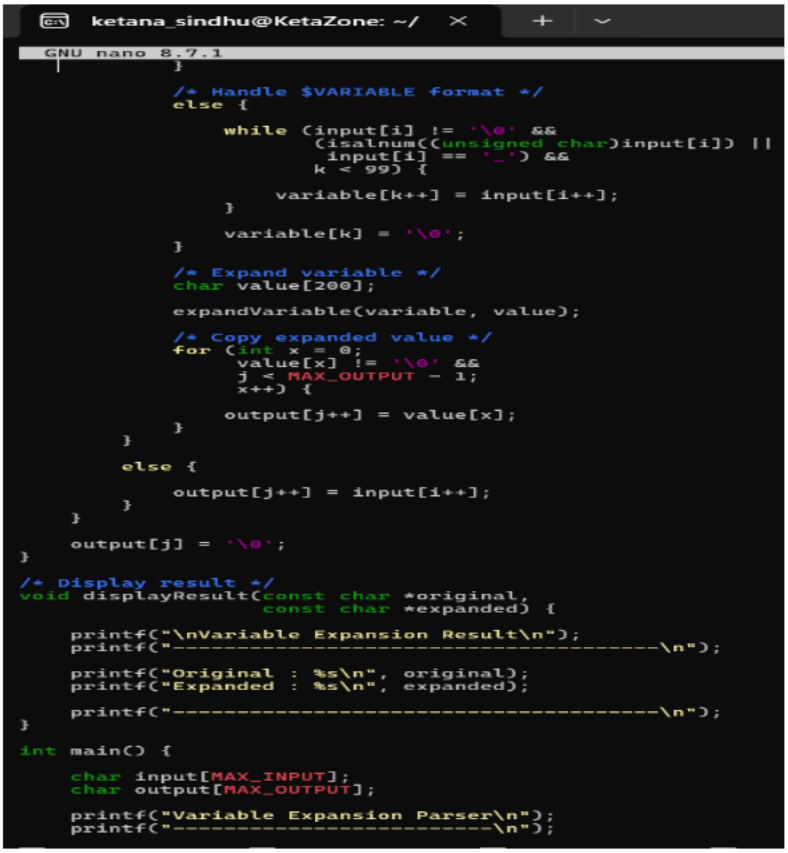
                while (input[i] != '\0' &&
                       input[i] != '}' &&
                       k < 99) {

                    variable[k++] = input[i++];
                }

                variable[k] = '\0';

                /* Skip closing brace */
                if (input[i] == '}')
                    i++;
            }

            /* Handle $VARIABLE format */
            else {
```



```
ketana_sindhu@KetaZone: ~/
  GNU nano 8.7.1
            }

            /* Handle $VARIABLE format */
            else {

                while (input[i] != '\0' &&
                       (isalnum((unsigned char)input[i]) ||
                        input[i] == '_') &&
                       k < 99) {

                    variable[k++] = input[i++];
                }

                variable[k] = '\0';
            }

            /* Expand variable */
            char value[200];

            expandVariable(variable, value);

            /* Copy expanded value */
            for (int x = 0;
                 value[x] != '\0' &&
                 j < MAX_OUTPUT - 1;
                 x++) {

                output[j++] = value[x];
            }
        }

        else {

            output[j++] = input[i++];
        }
    }

    output[j] = '\0';
}

/* Display result */
void displayResult(const char *original,
                   const char *expanded) {

    printf("\nVariable Expansion Result\n");
    printf("-------------------------------------\n");

    printf("Original : %s\n", original);
    printf("Expanded : %s\n", expanded);

    printf("-------------------------------------\n");
}

int main() {

    char input[MAX_INPUT];
    char output[MAX_OUTPUT];

    printf("Variable Expansion Parser\n");
    printf("-------------------------\n");
```

```c
        else {
            output[j++] = input[i++];
        }
    }

    output[j] = '\0';
}

/* Display result */
bheeshma@SINGAM: *original,
                 const char *expanded) {

    printf("\nVariable Expansion Result\n");
    printf("--------------------------------------\n");

    printf("Original : %s\n", original);
    printf("Expanded : %s\n", expanded);

    printf("--------------------------------------\n");
}

int main() {

    char input[MAX_INPUT];
    char output[MAX_OUTPUT];

    printf("Variable Expansion Parser\n");
    printf("-------------------------\n");

    /*
     * Set sample variables for testing.
     */
    setenv("NAME", "Ketana", 1);
    setenv("CITY", "Hyderabad", 1);
    setenv("COURSE", "CSE", 1);

    printf("Available variables:\n");
    printf("NAME = %s\n", getenv("NAME"));
    printf("CITY = %s\n", getenv("CITY"));
    printf("COURSE = %s\n\n", getenv("COURSE"));

    printf("Enter input: ");

    fgets(input, sizeof(input), stdin);

    /* Remove newline */
    input[strcspn(input, "\n")] = '\0';

    /* Expand variables */
    expandVariables(input, output);

    /* Display result */
    displayResult(input, output);

    return 0;
}
```

Result:

```
ketana_sindhu@KetaZone: ~/

bheeshma@SINGAM:~/SKILL$ nano Skil          l8a.c
bheeshma@SINGAM:~/SKILL$ gcc Skill          8a.c -o s8a
bheeshma@SINGAM:~/SKILL$ ./s8a
Variable Expansion Parser
-------------------------
Available variables:
NAME = Ketana
CITY = Hyderabad
COURSE = CSE

Enter input: Hello $NAME

Variable Expansion Result
--------------------------------------
Original : Hello $NAME
Expanded : Hello Ketana
--------------------------------------
bheeshma@SINGAM:~/SKILL$ ./s8a
Variable Expansion Parser
-------------------------
Available variables:
NAME = Ketana
CITY = Hyderabad
COURSE = CSE

Enter input: I am $NAME from $CITY and I study $COURSE

Variable Expansion Result
--------------------------------------
Original : I am $NAME from $CITY and I study $COURSE
Expanded : I am Ketana from Hyderabad and I study CSE
--------------------------------------
bheeshma@SINGAM:~/SKILL$
```

To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic.

Code:

ketana_sindhu@KetaZone: ~/

GNU nano 8.7.1

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define MAX_INPUT 200
#define MAX_ARGS 20

/* Function pointer type for built-in commands */
typedef int (*BuiltinFunction)(char **args);

/* Shell state */
char current_directory[MAX_INPUT];

/* ---------------- BUILT-IN COMMANDS ---------------- */

/* cd command */
int builtin_cd(char **args) {

    if (args[1] == NULL) {
        printf("cd: missing argument\n");
        return 1;
    }

    if (chdir(args[1]) != 0) {
        perror("cd");
        return 1;
    }

    getcwd(current_directory, sizeof(current_directory));

    printf("Directory changed to: %s\n", current_directory);

    return 1;
}

/* pwd command */
int builtin_pwd(char **args) {

    (void)args;

    getcwd(current_directory, sizeof(current_directory));

    printf("%s\n", current_directory);

    return 1;
}

/* echo command */
int builtin_echo(char **args) {

    for (int i = 1; args[i] != NULL; i++) {
        printf("%s", args[i]);

        if (args[i + 1] != NULL)
            printf(" ");
    }

    printf("\n");
```

```c
/* echo command */
int builtin_echo(char **args) {

    for (int i = 1; args[i] != NULL; i++) {
        printf("%s", args[i]);

        if (args[i + 1] != NULL)
            printf(" ");
    }

    printf("\n");

    return 1;
}

/* help command */
int builtin_help(char **args) {

    (void)args;

    printf("\nBuilt-in Commands:\n");
    printf("  cd <directory>   - Change directory\n");
    printf("  pwd              - Display current directory\n");
    printf("  echo <text>      - Display text\n");
    printf("  help             - Display help\n");
    printf("  exit             - Exit shell\n");

    return 1;
}

/* exit command */
int builtin_exit(char **args) {

    (void)args;

    printf("Exiting shell...\n");

    return 0;
}

/* ---------------- DISPATCH TABLE ---------------- */

typedef struct {
    char *name;
    BuiltinFunction function;
} Builtin;

/* Dispatch table */
Builtin dispatchTable[] = {
    {"cd", builtin_cd},
    {"pwd", builtin_pwd},
    {"echo", builtin_echo},
    {"help", builtin_help},
    {"exit", builtin_exit}
};

int builtinCount =
    sizeof(dispatchTable) / sizeof(dispatchTable[0]);

/* ---------------- PARSE INPUT ---------------- */
```

```
 GNU nano 8.7.1
/* ---------------- PARSE INPUT ---------------- */

int parseInput(char *input, char **args) {

    int count = 0;

    char *token = strtok(input, " \t\n");

    while (token != NULL && count < MAX_ARGS - 1) {

        args[count++] = token;

        token = strtok(NULL, " \t\n");
    }

    args[count] = NULL;

    return count;
}

/* ---------------- COMMAND DISPATCH ---------------- */

int executeBuiltin(char **args) {

    if (args[0] == NULL)
        return 1;

    for (int i = 0; i < builtinCount; i++) {

        if (strcmp(args[0], dispatchTable[i].name) == 0) {

            /* Execute function from dispatch table */
            return dispatchTable[i].function(args);
        }
    }

    printf("Invalid command: %s\n", args[0]);
    printf("Type 'help' to see available commands.\n");

    return 1;
}

/* ---------------- MAIN ---------------- */

int main() {

    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    getcwd(current_directory, sizeof(current_directory));

    printf("Built-in Command Shell\n");
    printf("----------------------\n");
    printf("Type 'help' for available commands.\n\n");

    while (1) {

        printf("shell[%s]> ", current_directory);
        fflush(stdout);
```

```
}

/* ---------------- MAIN ---------------- */

int main() {

    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    getcwd(current_directory, sizeof(current_directory));

    printf("Built-in Command Shell\n");
    printf("----------------------\n");
    printf("Type 'help' for available commands.\n\n");

    while (1) {

        printf("shell[%s]> ", current_directory);
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL) {
            break;
        }

        /* Remove newline */
        input[strcspn(input, "\n")] = '\0';

        /* Ignore empty input */
        if (strlen(input) == 0)
            continue;

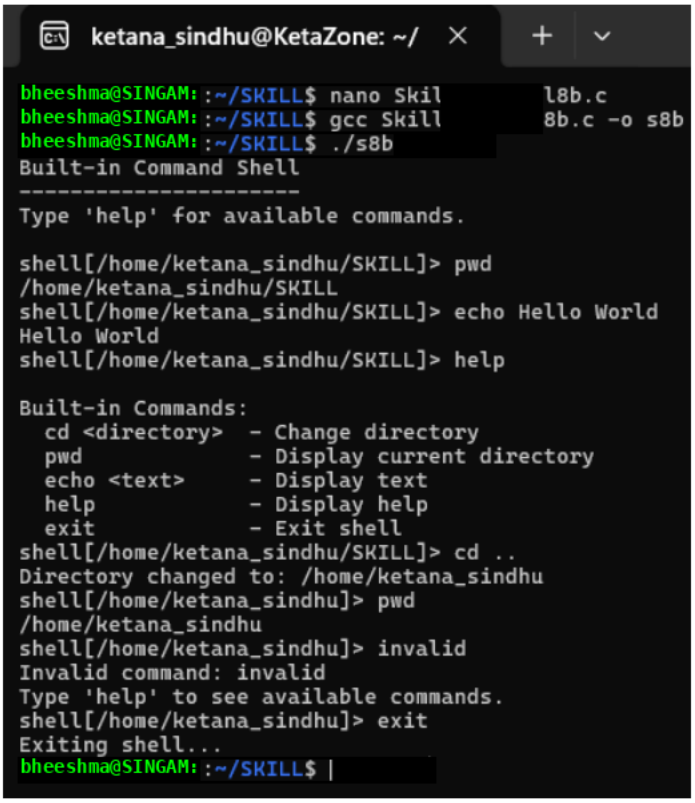
        /* Convert input into arguments */
        parseInput(input, args);

        /* Dispatch command */
        int result = executeBuiltin(args);

        /* Exit when command returns 0 */
        if (result == 0)
            break;
    }

    return 0;
}
```

Result:


ketana_sindhu@KetaZone: ~/
bheeshma@SINGAM:~/SKILL$ nano Skil l8b.c
bheeshma@SINGAM:~/SKILL$ gcc Skill 8b.c -o s8b
bheeshma@SINGAM:~/SKILL$ ./s8b
Built-in Command Shell
Type 'help' for available commands.
shell[/home/ketana_sindhu/SKILL]> pwd
/home/ketana_sindhu/SKILL
shell[/home/ketana_sindhu/SKILL]> echo Hello World
Hello World
shell[/home/ketana_sindhu/SKILL]> help
Built-in Commands:
cd <directory> - Change directory
pwd - Display current directory
echo <text> - Display text
help - Display help
exit - Exit shell
shell[/home/ketana_sindhu/SKILL]> cd ..
Directory changed to: /home/ketana_sindhu
shell[/home/ketana_sindhu]> pwd
/home/ketana_sindhu
shell[/home/ketana_sindhu]> invalid
Invalid command: invalid
Type 'help' to see available commands.
shell[/home/ketana_sindhu]> exit
Exiting shell...
bheeshma@SINGAM:~/SKILL$